Background

1) $Cov(X,Y) = E((X-\mu_X)(Y-\mu_Y)) = E(XY) - \mu_X\mu_Y$

2) $Cov(X,Y) = \frac{1}{N}\sum_{i=1}^{N}(x_i-\mu_x)(y_i-\mu_y)$, X,Y size N

$= \frac{1}{N}\sum_{i=1}^{N} x_i y_i - \mu_x\mu_y$

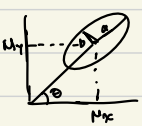
Normalized by $\frac{1}{N-1}$.

3) Cov Matrix for $X_1,\dots,X_n$ is Σ where $\Sigma(i,j) = Cov(x_i,x_j)$. So $\Sigma(i,i) = Cov(x_i,x_i) = Var(x_i)$. So $tr(\Sigma) \geq 0$. $\Sigma = \Sigma^T$ symmetric since $Cov(x,y) = Cov(y,x)$. And Σ positive semi definite since $\forall \vec{v} \in \mathbb{R}^n, \vec{v} \neq 0$ then $\vec{v}^T\Sigma\vec{v} \geq \vec{0}$. Hence eigen vals of $\Sigma \geq 0$.

4) $\vec{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}$, x_i r.v. with $Var(x_i) < \infty$, then $Cov(\vec{x}) = E((\vec{x}-\vec{\mu}_x)(\vec{x}-\vec{\mu}_x)^T)$, $\vec{\mu}_x = \begin{bmatrix} E(x_1) \\ \vdots \\ E(x_n) \end{bmatrix}$.

5) Multi dim normal dist. $p(\vec{x}\,|\,\vec{\mu},\Sigma) = \frac{1}{\sqrt{(2\pi)^n|\Sigma|}}\exp\left(-\frac{1}{2}(\vec{x}-\vec{\mu})^T\Sigma^{-1}(\vec{x}-\vec{\mu})\right)$, $\vec{x} \in \mathbb{R}^n$, $\vec{\mu} = E(\vec{x})$, $\Sigma \in M_{n\times n}(\mathbb{R})$ cov matrix.

6) Confidence ellipse



$\mu_x = E(x)$, $\mu_y = E(y)$, $\theta = \tan^{-1}\left(\frac{\mu_y}{\mu_x}\right)$, a highest eigenval square root of Cov matrix of X,Y, b second eigenval square root.

95% confidence ellipse has $2.45a$, $2.45b$.

7) Matrix B square root of A if $A = BB$.

8) Cholesky Decomp. If A positive definite, $A = L^TL$, lower triangular matrix. If A positive semidefinite, diag entries of L can be 0. $L(i,i) = \sqrt{A(i,i) - \sum_{j<i} L(i,j)^2}$ and $L(i,j) = \frac{1}{L(j,j)}\left(A(i,j) - \sum_{k<j} L(i,k)L(j,k)\right)$ for $i > j$

9) Jacobian $\frac{\partial f}{\partial x} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots \\ \vdots & \ddots \\ & \frac{\partial f_m}{\partial x_n} \end{bmatrix}$

$f(\vec{x}) = \begin{bmatrix} f_1(\vec{x}) \\ \vdots \\ f_m(\vec{x}) \end{bmatrix}$

1D Kalman Filter

read from textbook

1) $\vec{x}$ system state (true value)
$\hat{\vec{x}}_{n|m}$ estimate of system state at time n after m measurements.

2) Kalman filter optimality 1d. Have

both r.v.s with gaussian distribution

$$\hat{x}_{n|n} = w_1 z_n + (1-w_2)\hat{x}_{n|n-1}$$
$$p_{n|n} = w_1^2 r_n + (1-w_2)^2 p_{n|n-1}$$

r_n measurement variance. Then solving

$$\frac{dp_{n|n}}{dw_1} = 2w_1 r_n - 2(1-w_1)p_{n|n-1} = 0$$

$$\Rightarrow w_1 = \frac{p_{n|n-1}}{p_{n|n-1}+r_n} := K_n$$

3) Process noise q, if small lag error, if large follows measurements but estimate noisy.
↳ can scale q so your model, even if wrong still fits.

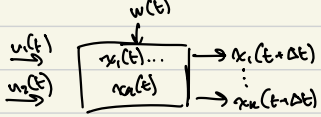
Multivariate Kalman Filter

1) State extrapolation equation: $\vec{x}_{n+1,n} = F\vec{x}_{n,n} + G\vec{u}_n + \vec{w}_n$, $\vec{x}_{n+1,n}$ predicted state at $t=n+1$, $\vec{x}_{n,n}$ estimated state at $t=n$, $\vec{u}_n$ control input or measured input, $\vec{w}_n$ process noise, F state transition matrix, G input transition matrix / control matrix

F: $k \times k$ matrix; not dim but time

$\vec{x}$: k length not neccesarily $m=k$

G: $k \times m$



2) LTI system representation $\dot{\vec{x}}(t) = A\vec{x}(t) + B\vec{u}(t)$, $\vec{y}(t) = C\vec{x}(t) + D\vec{u}(t)$.

A: system dynamic matrix; B: input matrix; C: output matrix; D: feedthrough matrix

set to this form from higher order ODE's by replacing higher order derivatives with vars.

$a_n \frac{d^n y}{dt^n} + \dots + a_1 \frac{dy}{dt} + a_0 y = u$. Let $x_1(t) = y, \dots, x_n(t) = \frac{d^{n-1} y}{dt^{n-1}}$, hence

$$\begin{bmatrix} \frac{dx_1}{dt} \\ \vdots \\ \frac{dx_n}{dt} \end{bmatrix} = \begin{bmatrix} \dot{x}_1 \\ \vdots \\ \dot{x}_n \end{bmatrix} = \underbrace{\begin{bmatrix} 0 & 1 & \cdots & 0 \\ \vdots & \ddots & \ddots & \vdots \\ 0 & \cdots & \cdots & 1 \\ -\frac{a_0}{a_n} & \cdots & \cdots & -\frac{a_{n-1}}{a_n} \end{bmatrix}}_{A} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} + u \underbrace{\begin{bmatrix} 0 \\ \vdots \\ 0 \\ \frac{1}{a_n} \end{bmatrix}}_{B}$$

3) LTI dynamic system to state extrapolation equation

$$\vec{x}(t+\Delta t) = e^{(A\Delta t)}\vec{x}(t) + \int_0^{\Delta t} e^{(At)}\, dt\, B\vec{u}(t)$$

A: matrix; Δt: times constant

where $e^C = \sum_{k=0}^{\infty} \frac{1}{k!} C^k$, C $n \times n$ matrix, $C^0 = I(n)$. Also $e^C = \lim_{k \to \infty} \left(I + \frac{C}{k}\right)^k$.

Also $\int_0^{\Delta t} e^{At}\, dt = \Delta t \left(I + \frac{A\Delta t}{2!} + \frac{(A\Delta t)^2}{3!} + \dots\right)$.

4) Covariance extrapolation equation: $P_{n+1,n} = F P_{n,n} F^T + Q$, $P_{n+1,n}$ is the squared uncertainty estimate (var in 1d case) cov matrix of a prediction of next state, $P_{n,n}$ is squared uncertainty of estimate (cov matrix) of current state, F state transition matrix, Q process noise matrix.

5) We know $Cov(\vec{x}) = E((\vec{x} - \vec{\mu}_x)(\vec{x} - \vec{\mu}_x)^T)$

So $P_{n,n} = E((\vec{x}_{n,n} - \hat{\vec{x}}_{n,n})(\vec{x}_{n,n} - \hat{\vec{x}}_{n,n})^T)$ (true value, estimate of state)

If $\hat{\vec{x}}_{n+1,n} = F\hat{\vec{x}}_{n,n} + G\vec{u}_{n,n}$ (no process noise), $P_{n+1,n} = E((\vec{x}_{n+1,n} - \hat{\vec{x}}_{n+1,n})(\vec{x}_{n+1,n} - \hat{\vec{x}}_{n+1,n})^T)$, subbing gives $P_{n+1,n} = F P_{n,n} F^T$.

6) With process noise, $\hat{\vec{x}}_{n+1,n} = F\hat{\vec{x}}_{n,n} + G\hat{\vec{u}}_{n,n} + \vec{w}_n$, Q can be diagonal if noise independent but can also not.
↳ discrete noise model → noise can change between measurements but constant inbetween.
↳ continuous noise model.

For discrete noise model if control/measured input not used in dynamic model, hence not used in F, give random variance for process noise $\vec{\sigma}_a^2$, $Q = F Q_a F^T$, Q_a = initial cov matrix for process noise. Eg. $Q_a = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & \sigma_a^2 \end{bmatrix}$ (no noise in velocity, displacement ↳ acceleration noise here not vec.)
Cause those measured and not used in F.
↳ If something with noise used in dynamics equation, can get $Q = G\vec{\sigma}_a^2 G^T$. (↳ i.e. control input)

For continuous model just get $Q_c = \int_0^{\Delta t} Q\,dt$, Q discrete process noise cov matrix.

Choice here, try both cts and discrete process noise model.

we only see this

7) Measurement equation: $\vec{z}_n = H\vec{x}_n + \vec{v}_n$, $\vec{z}_n$ measurement, $\vec{x}_n$ true system state (hidden), $\vec{v}_n$ random noise vector, H observation matrix. H has below use cases.

- Scaling raw input
- State selection, e.g. certain states measured while others not.
- Can only measure combination of states. e.g. len of triangle sides states but only perimeter measured.

8) State Update Equation: $\hat{\vec{x}}_{n,n} = \hat{\vec{x}}_{n,n-1} + K_n\left(\vec{z}_n - H\hat{\vec{x}}_{n,n-1}\right)$

diff in prediction and measurement (innovation)

observation matrix to make $\hat{x}$ form of z_n

Kalman gain brings dimensions back to state ones, updating state based on new knowledge

measurement domain → state domain

check kf optimal kalman gain derivation example.

Covupdate Equation:

9) $P_{n,n} = (I - K_nH)P_{n,n-1}(I - K_nH)^T + K_nR_nK_n^T$, $R_n = E(\vec{v}_n\vec{v}_n^T)$ measurement noise cov matrix

Assume → vec from now on

derivation: $\hat{x}_{n,n} = \hat{x}_{n-1,n} + K_n(z_n - H\hat{x}_{n,n-1})$

$= \hat{x}_{n-1,n} + K_n(Hx_n + v_n - H\hat{x}_{n,n-1})$

$e_n = x_n - \hat{x}_{n,n}$

$= x_n - \hat{x}_{n,n-1} - K_n(Hx_n + v_n - H\hat{x}_{n,n-1})$

$= x_n - \hat{x}_{n,n-1} - K_nHx_n - K_nv_n + H\hat{x}_{n,n-1}$

$= (I - K_nH)(x_n - \hat{x}_{n,n-1}) - K_nv_n$

$P_{n,n} = E(e_ne_n^T)$

$\vdots$

$= (I - K_nH)P_{n,n-1}(I - K_nH)^T + K_nR_nK_n^T$

1d case $\frac{P_{n,n-1}}{P_{n,n-1}+r_n}$ if $H=I$

10) Kalman Gain: $K_n = P_{n,n-1}H^T\left(HP_{n,n-1}H^T + R_n\right)^{-1}$

To minimize variance, need to minimize $tr(P_{n,n})$. Hence,

$$tr(P_{n,n}) = tr(P_{n,n-1}) - tr$$

$$\vdots$$

setting to 0 solving for K_n

$$\vdots$$

11) Simplified Cov update equation: $P_{n,n} = (I - K_nH)P_{n,n-1}$. Works well but errors in computing K_n can lead to huge computation errors since $I - K_nH$ can lead to non-symmetric matrices due to FP errors. Unstable equation.

Worked LTI Kalman Filter Example

Rocket altitude, has altimeter for altitude measurements and accelerometer for measuring rocket acceleration.

Accelerometer is the control input (determined by us, e.g. how much fuel is).

Let $\vec{x}_n = \begin{bmatrix} x_n \\ \dot{x}_n \end{bmatrix}$, $\vec{u}_n = [a_n + g]$ where $a_n = \ddot{x}_n - g + \epsilon_1$ (since accelerometer measures g as well; ϵ_1 ← acceleration measurement error)

Hence,

$$\begin{bmatrix} \hat{x}_{n+1,n} \\ \hat{\dot{x}}_{n+1,n} \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix}}_{F} \begin{bmatrix} \hat{x}_{n,n} \\ \hat{\dot{x}}_{n,n} \end{bmatrix} + \underbrace{\begin{bmatrix} 0.5\Delta t^2 \\ \Delta t \end{bmatrix}}_{G} [a_n + g]$$

using $Q = G\sigma_a^2 G^T$ derivation since 1 a control input

$$\rightarrow \quad Q = G[\epsilon^2]G^T$$

($[\epsilon^2]$: cov matrix of accelerometer (can have more if more control states); only rearrange in this case specifically)

$$= [\epsilon^2] G G^T$$

$$= \epsilon^2 \begin{bmatrix} \frac{\Delta t^4}{4} & \frac{\Delta t^3}{2} \\ \frac{\Delta t^3}{2} & \Delta t^2 \end{bmatrix}$$

(not system's random variance but rather control variance $\forall$ of r.v. a)

Can also derive since $\mathrm{Var}(\Delta x) = \mathrm{Var}(\frac{1}{2} a \Delta t^2) = (\frac{1}{2}\Delta t^2)^2 \mathrm{Var}(a) = \frac{\Delta t^4}{4}\epsilon^2$

$\mathrm{Var}(\Delta v) = \mathrm{Var}(a\Delta t) = \Delta t^2 \mathrm{Var}(a) = \Delta t^2 \epsilon^2$

$$\begin{aligned} \mathrm{Cov}(\Delta x, \Delta v) &= E(\Delta x \Delta v) - \mu_{\Delta x}\mu_{\Delta v} \\ &= E(\tfrac{1}{2} a \Delta t^2 a \Delta t) - E(\tfrac{1}{2} a \Delta t^2) E(a \Delta t) \\ &= \tfrac{1}{2} t^3 E(a^2) - \tfrac{1}{2} t^3 E(a) E(a) \\ &= \tfrac{1}{2} t^3 \left(E(a^2) - (E(a))^2 \right) \\ &= \tfrac{1}{2} t^3 \epsilon^2 \end{aligned}$$

Hence,

$$P_{n+1,n} = F P_{n,n} F^T + Q$$

Now for $\vec{z}_n = H\vec{x}_n + \vec{v}_n$, measurement only provides altitude x_n, hence, $H = [1\ 0]$.

Also $R_n = [\sigma^2_{x_m}]$ variance of altitude measurement, subscript m indicating measurement. Assume constant measurement var.

So,

$$K_n = P_{n,n-1}H^T(HP_{n,n-1}H^T + R_n)^{-1}$$

(cov and vars)

$$= \begin{bmatrix} P_{x_{n,n-1}} & P_{\dot{x}x_{n,n-1}} \\ P_{x\dot{x}_{n,n-1}} & P_{\dot{x}_{n,n-1}} \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} \left([1\ 0] \begin{bmatrix} \cdots \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} + [\sigma^2_{x_m}] \right)^{-1}$$

$$= \begin{bmatrix} P_{x_{n,n-1}} \\ P_{x\dot{x}_{n,n-1}} \end{bmatrix} \cdot \left[\frac{1}{P_{x_{n,n-1}} + \sigma^2_{x_m}} \right]$$

And finally,

$$P_{n,n} = (I - K_n H)P_{n,n-1}(I - K_n H)^T + K_n R_n K_n^T$$

For numeric example can assume constant acceleration $\ddot{x} = 30 m/s^2$, $\Delta t = 0.25$, $\sigma_{x_m} = 20m$, $\epsilon = 0.1 m/s^2$, and can start with initialization $\vec{x}_{0,0} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$, $\vec{v}_0 = [0]$, $P_{0,0} = \begin{bmatrix} 500 & 0 \\ 0 & 500 \end{bmatrix}$. Then go to first iteration and compute.

EKF

1) Kalman Filter assumes dist of all r.v.s is gaussian but for nonlinear systems, even for gaussian inputs, outputs not guaranteed to have gaussian dist.

2) State to measurement nonlinear → e.g. 

$x = d \cdot \tan\theta$. $H \to h$

3) Non linear system dynamics. $F \to f$

4) $f(x) \approx f(x_0) + f'(x_0)(x - x_0)$ $|x - x_0|$ small. EKF performs this analytic linearization. For f, h funcs, we only need to calculate $\frac{df(x)}{dx}$, $\frac{dh(x)}{dx}$.

5) In 1d, for points $(x_0 + \sigma_x, x_0 - \sigma_x)$, $y - \sigma_y = m(x_0 - \sigma_x) + b$, $y + \sigma_y = m(x_0 + \sigma_x) + b$, then $(y + \sigma_y) - (y - \sigma_y) = \ldots \Rightarrow \sigma_y = m\sigma_x$, $\sigma_y = \left(\frac{df(x)}{dx}\right)\sigma_x$. Hence for variance $p_y = \left(\frac{df(x)}{dx}\right)^2 p_x$.

So to linearise, in 1d case, just find $\frac{df(x)}{dx}$ or $\frac{dh(x)}{dx}$ and use as F or H matrix.

6) In n-dimensional case, $P_{out} = \left(\frac{\partial f}{\partial x}\right) P_{in} \left(\frac{\partial f}{\partial x}\right)^T$, for input covariance matrix P_{in}, P_{out} projected covariance, $\frac{\partial f}{\partial x}$ state transition jacobian. This is multivariate analytical linearization projection.

Same for state to measurement, replace f with h.

7) Update EKF equations.

State extrapolation: $\vec{x}_{n+1,n} = f(\vec{x}_{n,n}) + G\vec{u}_n + \vec{w}_n$

Covariance extrapolation: $P_{n+1,n} = \frac{\partial f}{\partial x} P_{n,n} \frac{\partial f}{\partial x}^T + Q$

State update: $\vec{x}_{n,n} = \vec{x}_{n,n-1} + K_n(\vec{z}_n - h(\vec{x}_{n,n-1}))$

Covariance update: $P_{n,n} = (I - K\frac{\partial h}{\partial x})P_{n,n-1}(I - K_n\frac{\partial h}{\partial x})^T + K_n R_n K_n^T$

Kalman Gain: $K_n = P_{n,n-1}\frac{\partial h}{\partial x}^T \left(\frac{\partial h}{\partial x} P_{n,n-1} \frac{\partial h}{\partial x}^T + R_n\right)^{-1}$

F, H linearized analytically so uncertainty PDF remains gaussian

8) EKF good when measurement/state transition close to linear. Otherwise use UKF.

9) Multirate Kalman Filter + Sensor Fusion (Todo)

Examples of EKF

Ex) Balloon →  $x = d\tan\theta$, $h(x_n) = \theta = \tan^{-1}\frac{x_n}{d}$. Linearize.

$\frac{dh(x_n)}{dx} = \frac{d}{d^2+x^2}$, hence $H = \left[\frac{d}{d^2+x^2}\right]$. Now perform LKF.

Ex) Car $\vec{z}_n = \begin{bmatrix} r_n \\ \varphi_n \end{bmatrix}$, $\vec{x}_n = \begin{bmatrix} x_n \\ y_n \end{bmatrix}$, polar coordinates. Hence to switch domains, $r = \sqrt{x^2+y^2}$, $\varphi = \tan^{-1}\left(\frac{y}{x}\right)$. $\vec{z}_n = h(\vec{x}_n) = \begin{bmatrix} \sqrt{x^2+y^2} \\ \tan^{-1}\left(\frac{y}{x}\right) \end{bmatrix}$. Hence,

$$\frac{\partial h}{\partial \vec{x}} = \begin{bmatrix} \frac{\partial(\sqrt{x^2+y^2})}{\partial x} & \frac{\partial(\sqrt{x^2+y^2})}{\partial y} \\ \vdots & \vdots \end{bmatrix}$$

$$= \begin{bmatrix} \frac{x}{\sqrt{x^2+y^2}} & \frac{y}{\sqrt{x^2+y^2}} \\ \frac{y}{x^2+y^2} & \frac{x}{x^2+y^2} \end{bmatrix}$$

Ex) Pendulum 

Rope applies force up hence $|F| = |mg\sin(\theta)| = |ma| \Rightarrow |a| = -g\sin\theta$. Also $s = L\theta$ (radians), s arc length. Hence $v = \frac{ds}{dt} = L\frac{d\theta}{dt}$. So $a = L\frac{d^2\theta}{dt^2} = -g\sin\theta$. Hence for state transition not matrix now, f is func of both $\vec{x}$ and $\vec{u}$ ← any measured control input.

$\vec{x}_{n+1,n} = f(\vec{x}_{n,n}, \vec{u}_n, \vec{w}_n)$ ← process noise

$$= \begin{bmatrix} \hat{\theta}_{n,n} + \hat{\dot{\theta}}_{n,n}\Delta t \\ \hat{\dot{\theta}}_{n,n} - \frac{g}{L}\sin(\hat{\theta}_{n,n})\Delta t \end{bmatrix}$$

← since $\Delta v = a\Delta t$

If we make state, $\vec{x}_n = [\theta_n]$, state to measurement linear, but if state $\vec{x}_n = [x_n]$, state-to-measurement also nonlinear.

Summary

prediction

1) $\hat{\vec{x}}_{n+1|n} = F\hat{\vec{x}}_{n|n} + G\vec{u}_n + \vec{w}_n$
2) $P_{n+1|n} = F P_{n|n} F^T + Q$
3) $\vec{z}_n = H\vec{x}_n + \vec{v}_n$
4) $R_n = E(\vec{v}_n \vec{v}_n^T)$ ← can compute diagonal of equipment, and rest 0 if with variance of $\vec{v}_n$ i.e. not correlated.
5) $Q_n = E(w_n w_n^T)$
6) $P_{n|n} = E(e_n e_n^T) = E((\vec{x}_n - \hat{\vec{x}}_{n|n})(\vec{x}_n - \hat{\vec{x}}_{n|n})^T)$

cov matrix of measurement uncertainty

cov matrix of process noise uncertainty

cov of estimation error

don't have $\vec{v}_n$, $\vec{w}_n$, $\vec{x}_n$ available to calculate directly

Kalman tries to minimize.

update

7) $K_n = P_{n|n-1} H^T (H P_{n|n-1} H^T + R_n)^{-1}$
8) $\hat{\vec{x}}_{n|n} = \hat{\vec{x}}_{n|n-1} + K_n(\vec{z}_n - H\hat{\vec{x}}_{n|n-1})$
9) $P_{n|n} = (I - K_n H) P_{n|n-1} (I - K_n H)^T + K_n R_n K_n^T$

Look at vec and matrix dims in summary.
Look at examples at end as well.

$\vec{z}_n$ v.s. $\vec{u}_n$ example, $\vec{z}_n$ only what we observe, $\vec{u}_n$ what we control.

EKF

State extrapolation: $\hat{\vec{x}}_{n+1|n} = f(\hat{\vec{x}}_{n|n}, \vec{u}_n) + \vec{w}_n$ ← can separate out $G\vec{u}_{n|n}$ here

Covariance extrapolation: $P_{n+1|n} = \frac{\partial f}{\partial \vec{x}} P_{n|n} \frac{\partial f}{\partial \vec{x}}^T + Q$ ← GQG^T here?

State update: $\hat{\vec{x}}_{n|n} = \hat{\vec{x}}_{n|n-1} + K_n(\vec{z}_n - h(\hat{\vec{x}}_{n|n-1}))$

Covariance update: $P_{n|n} = (I - K\frac{\partial h}{\partial \vec{x}}) P_{n|n-1} (I - K_n \frac{\partial h}{\partial \vec{x}})^T + K_n R_n K_n^T$

Kalman Gain: $K_n = P_{n|n-1} \frac{\partial h^T}{\partial \vec{x}} (\frac{\partial h}{\partial \vec{x}} P_{n|n-1} \frac{\partial h^T}{\partial \vec{x}} + R_n)^{-1}$

Pitch and Roll from gyro + accelerometer

Let $\vec{x} = \begin{bmatrix}\phi\\ \theta\end{bmatrix}$, ϕ roll, θ pitch. Gyro control input gives euler rate $\vec{u} = \begin{bmatrix}w_x\\ w_y\\ w_z\end{bmatrix}$. Accelerometer gives measurements $\vec{z} = \begin{bmatrix}a_x\\ a_y\\ a_z\end{bmatrix}$. Currently, assuming UAV at rest hence only gravity applied. Then, for $\vec{z} = h(\vec{x}) + \vec{v}_n$ where

$$h(\vec{x}) = h\left(\begin{bmatrix}\phi\\ \theta\end{bmatrix}\right) = \begin{bmatrix}-g\sin\theta\\ g\sin\phi\cos\theta\\ g\cos\phi\cos\theta\end{bmatrix} = \begin{bmatrix}a_x\\ a_y\\ a_z\end{bmatrix} = \vec{z}$$

h non linear so we have state-to-measurement nonlinearity. Now for the state transition function f,

$$\begin{aligned}\vec{\hat{x}}_{n+1,n} &= f(\vec{\hat{x}}_{n,n}, \vec{u}_{n,n}) + w_n\\ &= \begin{bmatrix}\hat{\phi} + \Delta t(w_x + w_y(\sin\hat{\phi}\tan\hat{\theta}) + w_z(\cos\hat{\phi}\tan\hat{\theta})\\ \hat{\theta} + \Delta t(w_y\cos\hat{\phi} - \sin\hat{\phi}\, w_z)\end{bmatrix}\end{aligned}$$

Note above $\hat{\theta}, \hat{\phi}$ are $\hat{\theta}_{n,n}, \hat{\phi}_{n,n}$ in $\vec{\hat{x}}_{n,n}$ respectively. Note that f also non-linear so we have state dynamics nonlinearity.

Hence for both nonlinearities we need to compute the Jacobian to update P, our uncertainty covariance matrix.

$$H = \frac{\partial h}{\partial \vec{x}} = \begin{bmatrix}0 & -g\cos\theta\\ g\cos\phi\cos\theta & -g\sin\phi\sin\theta\\ -g\sin\phi\cos\theta & -g\cos\phi\sin\theta\end{bmatrix}$$

$$F = \frac{\partial f}{\partial \vec{x}} = \begin{bmatrix}1 + \Delta t(w_y\cos\phi\tan\theta - w_z\sin\phi\tan\theta & \Delta t(w_y\sin\phi\sec^2\theta + w_z\cos\phi\sec^2\theta)\\ \Delta t(-w_y\sin\phi - w_z\cos\phi & 1\end{bmatrix}$$

Now for process noise matrix $Q = \alpha I_2$, and $R = \beta I_3, \alpha = 0.1, \beta = 1$ since we expect low process noise, and higher measurement error. Can tune α, β. Now just plug these values into EKF equations.

State extrapolation: $\vec{\hat{x}}_{n+1,n} = f(\vec{\hat{x}}_{n,n}, \vec{u}_{n,n}) + \vec{w}_n$

Covariance extrapolation: $P_{n+1,n} = \frac{\partial f}{\partial \vec{x}} P_{n,n} \frac{\partial f}{\partial \vec{x}}^T + Q$

State update: $\vec{\hat{x}}_{n,n} = \vec{\hat{x}}_{n,n-1} + K_n(\vec{z}_n - h(\vec{\hat{x}}_{n,n-1}))$

Covariance update: $P_{n,n} = (I - K\frac{\partial h}{\partial \vec{x}})P_{n,n-1}(I - K_n\frac{\partial h}{\partial \vec{x}})^T + K_n R_n K_n^T$

Kalman Gain: $K_n = P_{n,n-1}\frac{\partial h}{\partial \vec{x}}^T\left(\frac{\partial h}{\partial \vec{x}} P_{n,n-1}\frac{\partial h}{\partial \vec{x}}^T + R_n\right)^{-1}$

x,y,z for each

To integrate GPS + Magnometer, extend state to displacement, vel, acceleration, yaw and have GPS measure displacement, magnometer for yaw, accelerometer measures acceleration. Update pitch, roll calcs for acceleration measured (non constant location).

Model here assumes constant acceleration, to counteract we can make process noise for fields in state higher. Five have this.

using new state

Alternative is to have accelerometer in both control and measurement phase. Not sure how to do this tho.

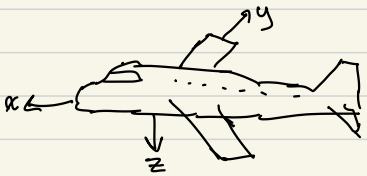
Magnetometer diff variance ← measured noise
Look into quarternions to linearize system dynamics.
Ask what should be in state v.s. control, what sensors measured v.s. control.
Computational pitfalls and best practice for trig funcs.
Multi rate sensors integration.

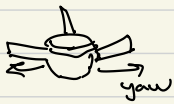
Solid ask

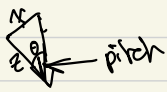
Next add yaw with compass.

Then GPS.

Roll, pitch, yaw and flight coords







$a_{xc} = -g\sin\theta$

$a_y = g\sin\phi\cos\theta$

$a_z = g\cos\phi\cos\theta$ $\quad \because g\cos 0\cos 0 = g$

Body rates to euler rates.

Attitude From Gyro, Accelerometer, and Magnetometer

Let $\vec{x} = \begin{bmatrix} \phi \\ \theta \\ \psi \end{bmatrix}$, ϕ roll, θ pitch, ψ yaw. Gyro control input $\vec{u} = \begin{bmatrix} w_x \\ w_y \\ w_z \end{bmatrix}$ body frame rates. To convert body rates to euler rates as before,

3-2-1 euler angles. NED frame

Magnetometer → δ true north v.s. real north, can get from GPS, latitude, longitude, set to 0 for now.

IGRF needed
on WMM
↖ check calibration diffs.

3 cases → data for acc, mag, both, seperate & concat vertically

GPS → accel twice $\begin{matrix} a_x \\ \vdots \\ a_z \end{matrix}$ | for roll/pitch other for state accel predicting (if not in control input) probably should be cause IMU data together

Better integration scheme (Improved Euler)

Attitude from Gyro, Accelerometer, Compass

Let $\vec{x} = \begin{bmatrix} \phi \\ \theta \\ \psi \end{bmatrix}$, ϕ roll, θ pitch, ψ yaw the 3-2-1 Euler angles, in NED frame. Gyro gives control inputs $\vec{u} = \begin{bmatrix} w_x \\ w_y \\ w_z \end{bmatrix}$, the body frame rates. To convert body frame rates to Euler rates we have

$$\begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix} = \begin{bmatrix} 1 & \sin\phi\tan\theta & \cos\phi\tan\theta \\ 0 & \cos\phi & -\sin\phi \\ 0 & \sin\phi\sec\theta & \cos\phi\sec\theta \end{bmatrix} \begin{bmatrix} w_x \\ w_y \\ w_z \end{bmatrix}$$

So integrating Euler rates, we have

$$\vec{x}_{n+1,n} = f(\vec{x}_{n,n}, \vec{u}_n) = \begin{bmatrix} \hat{\phi} + \Delta t(w_x + w_y \sin\hat{\phi}\tan\hat{\theta} + w_z \cos\phi\tan\theta) \\ \hat{\theta} + \Delta t(w_y \cos\hat{\phi} - w_z \sin\hat{\phi}) \\ \hat{\psi} + \Delta t(w_y \sin\hat{\phi}\sec\hat{\theta} + w_z \cos\hat{\phi}\sec\hat{\theta}) \end{bmatrix}$$

our state transition function. Notice it's nonlinear so we need $\frac{\partial f}{\partial \vec{x}}$, i.e. Jacobian for error covariance matrix calculations.

Now assuming a multirate EKF, whenever we get gyro data, we run the predict equations for EKF. When we get accelerometer data or magnetometer data, we run the update EKF equations with the associated observation function.

Accelerometer data: $\vec{z} = \vec{z}_{accel} = \begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix}$ NED accelerations. Still only assuming gravity is acting on UAV, hence our observation function is as follows.

$$h_{accel}(\hat{x}) = \begin{bmatrix} -g\sin\hat{\theta} \\ g\sin\hat{\phi}\cos\hat{\theta} \\ g\cos\hat{\phi}\cos\hat{\theta} \end{bmatrix} = \begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix}_{predicted}$$

Notice v not used here, hence it's column in the Jacobian $\frac{\partial h_{accel}}{\partial \vec{x}}$ will be $\vec{0}$, hence no update to v in $\vec{x}$ in this observation step.

Magnetometer data: $\vec{z}_{mag} = \begin{bmatrix} m_x \\ m_y \\ m_z \end{bmatrix}$ in the body frame. We construct $h_{mag}(\vec{x})$ which takes 3-2-1 Euler angles in NED frame to body frame we need rotation matrix R^{NED}_{body}.

$$R^{NED}_{body} = \begin{bmatrix} \cos\psi\cos\theta & \sin\psi\cos\theta & -\sin\theta \\ \cos\psi\sin\theta\sin\phi - \sin\psi\cos\phi & \sin\psi\sin\theta\sin\phi + \cos\psi\cos\phi & \cos\theta\sin\phi \\ \cos\psi\sin\theta\cos\phi + \sin\psi\sin\phi & \sin\psi\sin\theta\cos\phi - \cos\psi\sin\phi & \cos\theta\cos\phi \end{bmatrix}$$

Now notice NED coordinate frame is relative to true north, but the magnetometer measures the Earth's magnetic field relative to magnetic north. To fix this, we have to pull constant values $B_{NED} = \begin{bmatrix} B_x^{NED} \\ B_y^{NED} \\ B_z^{NED} \end{bmatrix}$, the Earth's magnetic field vector in global NED frame. We would have to calculate this beforehand from data such as inclination, declination, etc for our location. Pull this from IGRF or WMM. To predict $\vec{z} = \begin{bmatrix} m_x \\ m_y \\ m_z \end{bmatrix}$, we need to correct for this. Hence,

$$h_{mag}(\vec{x}) = R^{NED}_{body} B_{NED}$$

(B_{NED}: constant for location and altitude; R^{NED}_{body}: uses $\vec{x}$ internally)

Notice even though we intend to use magnetometer data to only correct ψ, it corrects ϕ, θ as well. Can maybe figure out a way to stop this.

Also error covariance matrices R_{accel}, R_{mag}, just set to $\epsilon^2_{accel} I, \epsilon^2_{mag} I$ initially.

In the case we get accelerometer and magnetometer data at the same time just let $h(\vec{x}) = \begin{bmatrix} h_{accel}(\vec{x}) \\ h_{mag}(\vec{x}) \end{bmatrix}$, $\vec{z} = \begin{bmatrix} \vec{z}_{accel} \\ \vec{z}_{mag} \end{bmatrix}$ and $R = \begin{bmatrix} R_{accel} & 0 \\ 0 & R_{mag} \end{bmatrix}$.

Also note that for $\theta \approx \frac{\pi}{2}$, we can expect errors cause of $\sec(\theta)$ terms. This is called gimbal lock and to solve this we need to switch to quaternion based EKF.

Quaternions Notes

1) Quaternion $\vec{q} = \begin{bmatrix} q_w \\ q_x \\ q_y \\ q_z \end{bmatrix}$ represents a 3-d rotation in space.

2) NED: North is x, East is y, Down is z.

3) $\vec{q}_{NED}$ to 3-2-1 euler angles $\begin{bmatrix} \phi \\ \theta \\ \psi \end{bmatrix}$, ϕ roll, θ pitch, ψ yaw.

$\phi = \arctan2\left(2(q_w q_x + q_y q_z),\ 1 - 2(q_x^2 + q_y^2)\right)$

$\theta = \arcsin\left(2(q_w q_y - q_z q_x)\right)$ ← must be clamped between $[-1, 1]$, otherwise gimbal lock

$\psi = \arctan2\left(2(q_w q_z + q_x q_y),\ 1 - 2(q_y^2 + q_z^2)\right)$

4) Rotation matrix to rotate $v \in \mathbb{R}^3$ through quaternion $\vec{q} = \begin{bmatrix} q_w \\ q_x \\ q_y \\ q_z \end{bmatrix}$,

$$R^{body}_{NED}(\vec{q}) = \begin{bmatrix} 1 - 2s(q_y^2 + q_z^2) & 2s(q_x q_y - q_w q_z) & 2s(q_x q_z + q_w q_y) \\ 2s(q_x q_y + q_w q_z) & 1 - 2s(q_x^2 + q_z^2) & 2s(q_y q_z - q_w q_x) \\ 2s(q_x q_z - q_w q_y) & 2s(q_y q_z + q_w q_x) & 1 - 2s(q_x^2 + q_y^2) \end{bmatrix} = \left(R^{NED}_{body}(\vec{q})\right)^T$$

$R^{NED}_{body} = C_i^b$

where $s = \|q\|^{-2}$, $s = 1$ if $\|\vec{q}\| = 1$, i.e. $\vec{q}$ unit quaternion, a pure rotation.

5) Quaternion multiplication. For $\vec{q}_1 = \begin{bmatrix} w_1 \\ x_1 \\ y_1 \\ z_1 \end{bmatrix}$, $\vec{q}_2 = \begin{bmatrix} w_2 \\ x_2 \\ y_2 \\ z_2 \end{bmatrix}$,

$$\vec{q}_1 \otimes \vec{q}_2 = \begin{bmatrix} w_1 w_2 - x_1 x_2 - y_1 y_2 - z_1 z_2 \\ w_1 x_2 + x_1 w_2 + y_1 z_2 - z_1 y_2 \\ w_1 y_2 - x_1 z_2 + y_1 w_2 + z_1 x_2 \\ w_1 z_2 + x_1 y_2 - y_1 x_2 + z_1 w_2 \end{bmatrix} = \begin{bmatrix} w_1 & -x_1 & -y_1 & -z_1 \\ x_1 & w_1 & -z_1 & y_1 \\ y_1 & z_1 & w_1 & -x_1 \\ z_1 & -y_1 & x_1 & w_1 \end{bmatrix} \vec{q}_2 = \begin{bmatrix} w_2 & -x_2 & -y_2 & -z_2 \\ x_2 & w_2 & y_2 & -z_2 \\ y_2 & -z_2 & w_2 & x_2 \\ z_2 & y_2 & -x_2 & w_2 \end{bmatrix} \vec{q}_1$$

$[\vec{q}_1]_L$ $[\vec{q}_2]_R$

Quaternion-based EKF for Attitude

Let $\vec{x} = \vec{q} = \begin{bmatrix} q_w \\ q_x \\ q_y \\ q_z \end{bmatrix}$ quaternion in NED frame for drone attitude. Gyro gives $\vec{u} = \begin{bmatrix} w_x \\ w_y \\ w_z \end{bmatrix}$ body frame rates. Body frame rates can be described as a quaternion $\vec{q}_w = \begin{bmatrix} 0 \\ w_x \\ w_y \\ w_z \end{bmatrix}$. Now note the quaternion differential equation is $\dot{\vec{q}} = \frac{1}{2}\vec{q} \otimes \vec{q}_w$, update for rotation to $\dot{\vec{x}} = \frac{1}{2}\vec{x} \otimes \vec{q}_w$, and to integrate $\vec{x}$ to get next state, we have

$$\hat{\vec{x}}_{n+1|n} = f(\hat{\vec{x}}_{n|n}, \vec{u}_n)$$

$$= \hat{\vec{x}}_{n|n} + \Delta t\, \dot{\vec{x}}$$

$$= \hat{\vec{x}}_{n|n} + \Delta t \left(\frac{1}{2}\hat{\vec{x}}_{n|n} \otimes \vec{q}_w\right)$$

$$= \begin{bmatrix} q_w + \frac{\Delta t}{2}(w_x q_x - w_y q_y - w_z q_z) \\ q_x + \frac{\Delta t}{2}(w_x q_w - w_y q_z + w_z q_y) \\ q_y + \frac{\Delta t}{2}(w_x q_z + w_y q_w - w_z q_x) \\ q_z + \frac{\Delta t}{2}(w_x q_y + w_y q_x + w_z q_w) \end{bmatrix}$$

f is our state extrapolation function. Now for measurement step.

Accelerometer: $\vec{z}_{acc} = \begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix}$ NED accelerations only assuming gravity. Can normalize these measurements by $\frac{1}{\sqrt{a_x^2 + a_y^2 + a_z^2}}$. Hence only assuming gravity our vector $\vec{g} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}^T$ (also normalized) in NED frame, and hence to get $\hat{\vec{z}}_{acc}$, we have to rotate $\vec{g}$ based on attitude of drone, i.e. through $\hat{\vec{x}}_{n|n-1}$. Hence,

we normalize so $\hat{\vec{x}}_{n|n}$ remains unit quaternion

$$\hat{\vec{z}}_{acc} = h(\hat{\vec{x}}_{n|n-1})$$

$$= R^{NED}_{body}(\hat{\vec{x}}_{n|n-1})\, \vec{g}$$

$$= \begin{bmatrix} 2(q_x q_z - q_w q_y) \\ 2(q_y q_z + q_w q_x) \\ 1 - 2(q_x^2 + q_y^2) \end{bmatrix}$$

R can be set to $\sigma_{acc}^2 I_3$.

Magnetometer: $\vec{z}_{mag} = \begin{bmatrix} m_x \\ m_y \\ m_z \end{bmatrix}$ in the body frame. We need to pull the Earth's magnetic field vector $\vec{r} = \begin{bmatrix} r_x \\ r_y \\ r_z \end{bmatrix}$, as before from WMM. Can calculate this with magnetic dip θ_{dip} so $\vec{r}_{NED} = \begin{bmatrix} \cos\theta_{dip} \\ 0 \\ \sin\theta_{dip} \end{bmatrix}$. Now to get $\vec{z}_{mag}$ from $\vec{r}_{NED}$, we have to rotate $\vec{r}_{NED}$ through $\hat{\vec{x}}_{n,n-1}$. Hence

$$\begin{aligned} \vec{z}_{mag} &= h(\hat{\vec{x}}_{n,n-1}) \\ &= R_{body}^{NED}(\hat{\vec{x}}_{n,n-1})\,\vec{r}_{NED} \end{aligned}$$

$$= \begin{bmatrix} r_x(1-2(q_y^2+q_z^2)) + r_z 2(q_x q_z - q_w q_y) \\ r_x 2(q_x q_y - q_w q_z) + r_z 2(q_y q_z + q_w q_x) \\ r_x 2(q_w q_y + q_x q_z) + r_z(1-2(q_x^2+q_y^2)) \end{bmatrix} \qquad \begin{aligned} r_x &= \cos\theta_{dip} \\ r_z &= \sin\theta_{dip} \end{aligned}$$

Note we normalize $\vec{z}_{mag}$ reading and $\vec{r}_{NED}$ already normalized. R can be set to $\sigma_{mag}^2 I_3$.

Multistep: As before $\vec{z} = \begin{bmatrix} \vec{z}_{acc} \\ \vec{z}_{mag} \end{bmatrix}$, $h(\hat{\vec{x}}_{n,n-1}) = \begin{bmatrix} h_{acc}(\hat{\vec{x}}_{n,n-1}) \\ h_{mag}(\hat{\vec{x}}_{n,n-1}) \end{bmatrix}$, and $R = \begin{bmatrix} R_{acc} & 0_3 \\ 0_3 & R_{mag} \end{bmatrix}$.

Now for process noise Q_t on each timestep, we compute $Q_t = \frac{\partial f}{\partial \vec{u}_t} \Sigma_\omega \frac{\partial f}{\partial \vec{u}_t}^T$, where $\Sigma_\omega = \sigma_{gyro}^2 I_3$. Same can be done for previos iterations of EKF.

Now there is a problem b/c we have to normalize $\hat{\vec{x}}_{n,n} = \hat{\vec{x}}_{n,n-1} + K_n(\vec{z}_n - h(\hat{\vec{x}}_{n,n-1}))$ by $\frac{1}{\|\hat{\vec{x}}_{n,n}\|}$ since $\|\hat{\vec{x}}_{n,n}\|$ must be 1. This makes this EKF more suboptimal, however can work in practice. To further optimize we can use the error state MEKF which uses multiplication for update step, maintaining normalization of state.

ES EKF notes

1) Acc control now → can't use if moving $\vec{v} - \vec{a} = 0$. Can use GPS as u, but bad.
2) Why not IMU observe? Only observe calls?
3) Wind → Stochastic Processes
4) GPS not for disp (maybe) } only diff ardupilot
5) IEKF, UEKF future
6) → new integration scheme / I + F dt + $F^2 \frac{\Delta t^2}{2}$ ... Taylor
7) Noise $\vec{q}$ estimation, $\vec{v}$ estimation, $\vec{r}$ estimation from trap int, $\delta \vec{x} = 0$ each iteration. But need prev P not this.
8) Assumes earth inertial frame
9) Wind → Measurement Cov (keep track of Δt only for GPS).
   ↳ or seperate sensor → sep measurement $(I - [\omega\times]) C(\hat{q})$ sub from v every where ↳ or update measurement cov param
10) GPS disp measure (correct and vel)? → leave out vel correction for now
    ↳ If vel how to get doppler / more accurate v for ≥. Seperate sensors? altimeter
11) Reset error state for each update → even if sensor not use err state (Multistate) ↳ cause
12) Bias estimate → update calibration of each sensor, or just accumulate in state vec. then set = 0

11) Model only assumes bias for gyro, mag, acc → GPS, wind sensor properly calibrated at flight start. e.g initial loc. GPS also give x,y,z disp rather than lat long.

12) Pull global earth vec from GPS for long dist?

13) Measurement only after predict → Flags for measurement ready or not.
↳ GPS store Δt_{GPS} accross GPS measurements for wind

14) Question of $\delta x_{n,n} = S_{.x} x_{n,n-1} + K_n(\text{innovation})$ ← does this update Eq? Check with code.

15) Normalization for floating point errors.

16) Driver update for bias calibration IMU, Magnetometer directly. → couples sensor fusion calls iface methods

17) Integration with zp, memory loc for nominal state data.

Error State Multiplicative EKF

Redefining notation

Body frame → rotates with drone, nose is x, right wing y, down is z.
Inertial, fixed frame (Earth) with NED corresponding to x,y,z here.
For vec $\vec{v}$, $\vec{v}^i$ in intertial frame, $\vec{v}^b$ body frame.

For quaternion $\vec{q} = \begin{Bmatrix} q_1 \\ \vec{q}_{2:4} \end{Bmatrix}$. Quaternion product can be rewritten as

$$\vec{p} \otimes \vec{q} = \begin{bmatrix} p_1 q_1 - \vec{p}_{2:4} \cdot \vec{q}_{2:4} \\ p_1 \vec{q}_{2:4} + q_1 \vec{p}_{2:4} + \vec{p}_{2:4} \times \vec{q}_{2:4} \end{bmatrix}$$

(dot product)

Inverse $\vec{q}^{-1} = \begin{bmatrix} q_1 \\ -\vec{q}_{2:4} \end{bmatrix}$. Identity $= \begin{bmatrix} 1 \\ \vec{0} \end{bmatrix}$. So for vector $\vec{v}^b$ in body frame $\begin{bmatrix} 0 \\ \vec{v}^b \end{bmatrix} = \vec{q}^{-1} \otimes \begin{bmatrix} 0 \\ \vec{v}^i \end{bmatrix} \otimes \vec{q}$. Same can be done with $\vec{v}^b = C_i^b(\vec{q}) \vec{v}^i$, C_i^b from i frame to b frame, same as R before (sub/superscripts may be inconsistent).

$$C_i^b(\vec{q}) = 2\vec{q}_{1:4}\vec{q}_{2:4}^T + I_3(q_1^2 - \vec{q}_{1:4}^T \vec{q}_{1:4}) - 2q_1[\vec{q}_{2:4}\times]$$

Also, for $\vec{v} \in V$, $\dim(V) = 3$, then

$$[\vec{v}\times] = \begin{bmatrix} 0 & -v_3 & v_2 \\ v_3 & 0 & v_1 \\ -v_2 & v_1 & 0 \end{bmatrix}$$

skew symmetric matrix so that $[\vec{v}\times]\vec{w} = \vec{v} \times \vec{w} \quad \forall \vec{v}, \vec{w}$.

Updated Kalman Filter Setup

Previously we had, gyro as control, accelerometer and magnetometer as measurements/correction. This only works for stationary objects as accelerometer only measures gravity, but in case of movement, can't construct measurement function because to estimate $\vec{q}$, we would need to know actual acceleration of drone but that is itself being measured from accelerometer.

As a result, can have both accelerometer and gyro as control and other sensors as measurement/correction. (can have all sensors as correction, but this way more common in literature, and makes sense since gyro + accelerometer have highest rate of data (IMU has most frequent measurements).

ES MEKF Setup

In the ES MEKF, rather than our EKF estimating our desired values (e.g. attitude, velocity, position) in it's state, in the EKF, we estimate the errors in a naive estimation of our state from our dynamics equation. E.g. we would naively integrate $\hat{q}$ from our $\dot{q}$ from our gyro input, and have $\delta\vec{q}$ in our EKF.

System Dynamics

We want to estimate $\vec{r}^i$; $\vec{v}^i$; $\vec{q}$, displacement, velocity, and quaternion for attitude. We get $\vec{\omega}^b_{ib}$, body-fixed coordinates of angular velocity of body frame w.r.t. the inertial frame from gyro and $\vec{f}^b$ acceleration from accelerometer. We have the following system dynamics equations

$$\dot{\vec{q}} = \frac{1}{2}\vec{q} \otimes \begin{bmatrix} 0 \\ \vec{\omega}^b_{ib} \end{bmatrix} \quad \cdots (1)$$

$$\dot{\vec{v}}^b = C^i_b(\vec{q})\vec{f}^b + \vec{g}^i \quad \cdots (2)$$

For (1),

$$\dot{\vec{q}} = \frac{1}{2}\begin{bmatrix} 0 & -\vec{\omega}^{b\,T}_{ib} \\ \vec{\omega}^b_{ib} & -[\vec{\omega}^b_{ib}\times] \end{bmatrix}\vec{q} \quad \because \text{rewrite quaternion mult matrix}$$

$$:= \frac{1}{2}\Omega(\vec{\omega}^b_{ib})\vec{q}$$

This is a linear equation hence recalling notes on the multivariate KF, to go from a dynamics system to state extrapolation, we get

$$\vec{q}_k = \exp\left(\tfrac{1}{2}\Omega(\vec{\bar{\omega}})\Delta t\right)\vec{q}_{k-1}$$

where

$$\vec{\bar{\omega}} = \frac{\omega_k + \omega_{k-1}}{2} \quad \dots (4)$$

Incremental rotation vec can be approximated as $\vec{\delta} = \frac{1}{2}\vec{\omega}\,\delta t$. The general solution to

$$\exp\left(\Omega(\vec{\delta})\right) = \cos(\|\vec{\delta}\|) I_4 + \frac{\sin(\|\vec{\delta}\|)}{\|\vec{\delta}\|}\Omega(\vec{\delta}) \qquad \text{(proven in appendix)}$$

Hence,

$$\begin{aligned}\vec{q}_k &= \exp\left(\tfrac{1}{2}\Omega(\vec{\bar{\omega}})\Delta t\right)\vec{q}_{k-1} \\ &= \exp\left(\Omega\left(\tfrac{1}{2}\Delta t\,\vec{\bar{\omega}}\right)\right)\vec{q}_{k-1} \\ &= \left(\cos\left(\tfrac{\Delta t}{2}\|\vec{\bar{\omega}}\|\right) I_4 + \frac{\sin\left(\frac{1}{2}\Delta t\|\vec{\bar{\omega}}\|\right)}{\frac{1}{2}\Delta t\|\vec{\bar{\omega}}\|}\tfrac{1}{2}\Delta t\,\Omega(\vec{\bar{\omega}})\right)\vec{q}_{k-1}\end{aligned}$$

$$\vec{q}_k = \left(\cos\left(\tfrac{\Delta t}{2}\|\vec{\bar{\omega}}\|\right) I_4 + \frac{\sin\left(\frac{\Delta t}{2}\|\vec{\bar{\omega}}\|\right)}{\|\vec{\bar{\omega}}\|}\Omega(\vec{\bar{\omega}})\right)\vec{q}_{k-1} \quad \dots (5)$$

Now for velocity, we can integrate acceleration (trapezoidal scheme chosen here).

$$\vec{v}_k = \frac{\Delta t}{2}\left(C_b^i(\vec{q}_k)\vec{f}_k^b + C_b^i(\vec{q}_{k-1})\vec{f}_{k-1}^b\right) + \vec{v}_{k-1} \quad \dots (6)$$

and position can similarly be integrated.

$$r_k^i = \frac{\Delta t}{2}\left(v_k^i + v_{k-1}^i\right) + r_{k-1}^i \quad \dots (7)$$

So now on each predict step, we can compute (4), (5), (6), (7), from ω_k and f_k, and now in the EKF, we can have the errors for each. Then on each measurement we can compute changes in errors and then update our nominal state. Note for trapezoidal integration, we have to compute $\vec{q}_k$ first.

Error State Formulation

IMU outputs have errors $\omega = \hat{\omega} + \delta\omega$, $f = \hat{f} + \delta f$ noise (dropping subscripts and super scripts from now on). $\delta\omega, \delta f$ random noise here. For quaternion q, we have small error quaternion δq for estimate $\hat{q}$ so

$$q = \hat{q} \otimes \delta q \iff \delta q = \hat{q}^{-1} \otimes q$$

Also, $$C_i^b(q) = C_i^b(\hat{q} \otimes \delta q) = C_i^b(\delta q)\, C_i^b(\hat{q})$$

Now by differentiating $\delta q = \hat{q}^{-1} \otimes q$, we get

$$\delta\dot{q} = \hat{q}^{-1} \otimes \dot{q} + \dot{\hat{q}}^{-1} \otimes q$$

and we can get an identity for $\dot{\hat{q}}^{-1}$ by differentiating $\begin{bmatrix} 1 \\ \vec{0} \end{bmatrix} = \hat{q}^{-1} \otimes \hat{q}$

$$\frac{d}{dt}\begin{bmatrix} 1 \\ \vec{0} \end{bmatrix} = \frac{d}{dt}\left[\hat{q}^{-1} \otimes \hat{q}\right]$$

$$\Rightarrow \vec{0} = \dot{\hat{q}}^{-1} \otimes \hat{q} + \hat{q}^{-1} \otimes \dot{\hat{q}}$$

$$\Rightarrow -\dot{\hat{q}}^{-1} \otimes \hat{q} = \hat{q}^{-1} \otimes \dot{\hat{q}}$$

$$\Rightarrow \dot{\hat{q}}^{-1} = -\hat{q}^{-1} \otimes \dot{\hat{q}} \otimes \hat{q}^{-1}$$

$$\Rightarrow \dot{\hat{q}}^{-1} = -\hat{q}^{-1} \otimes \frac{1}{2}\hat{q} \otimes \begin{bmatrix} 0 \\ \hat{\omega}_{ib} \end{bmatrix} \otimes \hat{q}^{-1} \qquad \text{from } ①$$

$$\Rightarrow \dot{\hat{q}}^{-1} = \frac{1}{2}\begin{bmatrix} 0 \\ \hat{\omega} \end{bmatrix} \otimes \hat{q}^{-1} \qquad \ldots ⑧$$

So,

$$\delta\dot{q} = \hat{\dot{q}}^{-1} \otimes \dot{q} + \hat{\dot{q}}^{-1} \otimes q$$

$$= \hat{\dot{q}}^{-1} \otimes \frac{1}{2} q \begin{bmatrix} 0 \\ w \end{bmatrix} + \hat{\dot{q}}^{-1} \otimes q \qquad \text{from } ①$$

$$= \hat{\dot{q}}^{-1} \otimes \frac{1}{2} q \begin{bmatrix} 0 \\ w \end{bmatrix} - \frac{1}{2} \begin{bmatrix} 0 \\ \hat{w} \end{bmatrix} \otimes \hat{\dot{q}}^{-1} \otimes q \qquad \text{from } ②$$

$$= \frac{1}{2} \delta q \otimes \begin{bmatrix} 0 \\ w \end{bmatrix} - \frac{1}{2} \begin{bmatrix} 0 \\ \hat{w} \end{bmatrix} \otimes \delta q \qquad \because \delta q = \hat{\dot{q}}^{-1} \otimes q$$

$$= \frac{1}{2} \delta q \otimes \begin{bmatrix} 0 \\ \hat{w} + \delta w \end{bmatrix} - \frac{1}{2} \begin{bmatrix} 0 \\ \hat{w} \end{bmatrix} \otimes \delta q \qquad \because w = \hat{w} + \delta w$$

$$= \frac{1}{2} \delta q \otimes \begin{bmatrix} 0 \\ \hat{w} \end{bmatrix} - \frac{1}{2} \begin{bmatrix} 0 \\ \hat{w} \end{bmatrix} \otimes \delta q + \frac{1}{2} \delta q \otimes \begin{bmatrix} 0 \\ \delta w \end{bmatrix} \qquad \because \text{quaternion mult not commutative}$$

Now we make an assumption that δq has small magnitude so $\delta q_1 \approx 1$, hence we don't estimate it.

$$\Rightarrow \delta\dot{q} = \frac{1}{2} \begin{Bmatrix} -\delta q_{2:4} \cdot \hat{w} \\ \hat{w} + \delta q_{2:4} \times \hat{w} \end{Bmatrix} + \frac{1}{2} \begin{Bmatrix} \hat{w} \cdot \delta q_{2:4} \\ -\hat{w} - \hat{w} \times \delta q_{2:4} \end{Bmatrix} + \frac{1}{2} \begin{Bmatrix} -\delta q_{2:4} \cdot \delta\hat{w} \\ \delta\hat{w} + \delta q_{2:4} \times \delta\hat{w} \end{Bmatrix} \qquad \because \text{quat mult definition}$$

$$\Rightarrow \begin{Bmatrix} 0 \\ \delta\dot{q}_{2:4} \end{Bmatrix} = \begin{Bmatrix} 0 \\ -\hat{w} \times \delta q_{2:4} \end{Bmatrix} + \frac{1}{2} \begin{Bmatrix} -\delta q_{2:4} \cdot \delta\tilde{w} \\ \delta\tilde{w} + \delta q_{2:4} \times \delta\tilde{w} \end{Bmatrix}$$

assuming magnitude not changing, since $\delta q_1 \approx 1$

2nd order term

We further approximate by assuming $\delta q_{2:4}$, $\delta\tilde{w}$ small, hence

$$\delta\dot{q}_{2:4} \approx -\hat{w} \times \delta q_{2:4} + \frac{1}{2} \delta\tilde{w}$$

We now write $\alpha = 2\delta q_{2:4} \ldots ⑨$, and get

$$\delta\dot{\alpha} \approx -\hat{w} \times \alpha + \delta w \qquad \ldots ⑩$$

We also get

$$C_i^b(\delta q) = 2\delta\vec{q}_{1:4}\delta\vec{q}_{2:4}^T + I_3(\delta q_1^2 - \delta\vec{q}_{1:4}^T\delta\vec{q}_{2:4}) - 2\delta q_1[\delta q_{2:4}\times].$$

Assuming 2nd order terms $0, \delta q_{1:3} \approx I$, we get

$$C_i^b(q) = (I - [\alpha\times])C_i^b(\hat{q}) \quad \ldots (11)$$

$$C_b^i(q) = C_b^i(\hat{q})[I + [\alpha\times]] \quad \ldots (12)$$

Now for velocity and position error dynamics, we have

$$\delta v = v - \hat{v}$$
$$\delta r = r - \hat{r}$$
$$\delta\dot{v} = \dot{v} - \dot{\hat{v}}$$

$$\Rightarrow \delta\dot{v} = C_b^i(q)f^b + g - C_b^i(\hat{q})\hat{f}^b - \hat{g} \quad \because (2)$$

$$\Rightarrow \delta\dot{v} = C_b^i(q)(\hat{f} + \delta f) - C_b^i(\hat{q})\hat{f} + \delta g \quad \because \delta = \hat{f} + \delta f$$

$$\Rightarrow \delta\dot{v} = C_b^i(\hat{q})[I + [\alpha\times]](\hat{f} + \delta f) - C_b^i(\hat{q})\hat{f} + \delta g \quad \because (12)$$

$$\Rightarrow \delta\dot{v} = C_b^i(\hat{q})[\alpha\times]\hat{f} + C_b^i(\hat{q})[I + [\alpha\times]](\delta f) + \delta g$$

$$\Rightarrow \delta\dot{v} = -C_b^i(\hat{q})[\hat{f}\times]\alpha + C_b^i(\hat{q})[I + [\alpha\times]](\delta f) + \delta g \quad \because [\alpha\times]\hat{f} = -[\hat{f}\times]\alpha$$
cross product identity

$$\Rightarrow \delta\dot{v} = -C_b^i(\hat{q})[\hat{f}\times]\alpha + C_b^i(\hat{q})\delta f \quad \ldots (13) \quad \because \text{assuming } \delta g \approx 0,\ C_b^i(\hat{q})[I + [\alpha\times]]\delta f \approx C_b^i(\hat{q})\delta f$$
with 2nd degree small term removed

Also finally $\delta\dot{r} = \dot{r} - \dot{\hat{r}} = v - \hat{v} = \delta v \quad \ldots (14)$.

Bias State Estimation

We assume bias only in gyro, accelerometer, magnetometer. All other sensors should be calibrated to remove biases and only have random white noise error.

Assume $\omega = \tilde{\omega} - B_\omega - n_\omega$, $\dot{B}_\omega = v_\omega$, n_ω, v_ω random noise with cov matrices

$$E[n_\omega(t) n_\omega^T(\tau)] = \begin{bmatrix} \sigma_{\omega^2} & 0 & 0 \\ 0 & \sigma_{\omega^2} & 0 \\ 0 & 0 & \sigma_{\omega^2} \end{bmatrix} \delta(t-\tau)$$

$$E[v_w(t) v_w^T(\tau)] = \begin{bmatrix} \sigma_{B\omega^2} & 0 & 0 \\ 0 & \sigma_{B\omega^2} & 0 \\ 0 & 0 & \sigma_{B\omega^2} \end{bmatrix} \delta(t-\tau)$$

Can be seperate values if var different for x,y,z axises

And $\delta\omega = -B_\omega - n_\omega$, $\dot{\delta\omega} = \omega - \hat{\omega}$, $\dot{\delta\omega} = -v_\omega$.

Similarly, $f = \tilde{\delta} - B_f - n_f$, $\dot{B}_f = v_f$, $\delta\dot{f} = -B_f - n_f$, with n_f, v_f random white noise, $E[n_f(t) n_f^T(\tau)] = \text{diag}(\sigma_f^2)\delta(t-\tau)$, $E[v_f(t) v_f^T(\tau)] = \text{diag}(\sigma_{B_f}^2)\delta(t-\tau)$.

Finally for magnetometer, bias term still used since some correlation expected between KF update equations. For B_m, model assumes it as a diverging process due to white noise, $\dot{B}_m = v_m$, v_m white noise, $E[v_m(t) v_m^T(\tau)] = \text{diag}(\sigma_{Bm}^2)\delta(t-\tau)$.

Predict Step Summary

1) Trapezoidal integration of v, r, and update q with ω.
2) EKF state $\delta x = \{\alpha^T \ \delta v^T \ \delta r^T \ B_\omega^T \ B_f^T \ B_m^T\}^T$ with

$$\begin{bmatrix} \dot{\alpha} \\ \dot{\delta v} \\ \dot{\delta r} \\ \dot{B}_\omega \\ \dot{B}_f \\ \dot{B}_m \end{bmatrix} = \begin{bmatrix} -[\hat{\omega}\times] & 0_3 & 0_3 & I_3 & 0_3 & 0_3 \\ -C_b^i(\hat{q})[\hat{f}^b\times] & 0_3 & 0_3 & 0_3 & -C_b^i(\hat{q}) & 0_3 \\ 0_3 & I_3 & 0_3 & 0_3 & 0_3 & 0_3 \\ & & & 0_{9\times 15} & & \end{bmatrix} \delta x + \begin{bmatrix} -n_\omega \\ -C_b^i(\hat{q}) n_f \\ 0_{3\times 1} \\ v_\omega \\ v_f \\ v_m \end{bmatrix}$$

$F(\hat{q}, \hat{\omega}, \hat{f})$

3) Process noise which can be generated assuming F linearized at $\vec{w}=\vec{\delta}=0$ and $C_i^b(\hat{q})=I$. Works well in practice.

$$Q_d = \int_0^{\Delta t} e^{F(t-\tau)} Q_c e^{F^T(t-\tau)} d\tau$$

where for $\dot{\delta x} = F\delta x + Gu$, $E[(Gw)(Gw)^T] = Q_c\,\delta(t-\tau)$, hence for white noise, Q_c diagonal with

$$Q_c = \begin{bmatrix} \text{diag}(\sigma_w^2) & & & & & \\ & \text{diag}(\sigma_f^2) & & & & \\ & & 0 & & & \\ & & & \text{diag}(\sigma_{Bw}^2) & & \\ & & & & \text{diag}(\sigma_{Bf}^2) & \\ & & & & & \text{diag}(\sigma_{Bm}^2) \end{bmatrix}$$

Some assumptions here for noise can be fixed but this should work well in practice. So,

$$Q_d = \begin{bmatrix} N(\sigma_w^2)\Delta t + N(\sigma_{Bw}^2)\frac{\Delta t^3}{3} & 0 & 0 & -N(\sigma_{Bw}^2)\frac{\Delta t^2}{2} & 0 & 0 \\ 0 & N(\sigma_f^2)\Delta t + N(\sigma_{Bf}^2)\frac{\Delta t^3}{3} & N(\sigma_{Bf}^2)\frac{\Delta t^4}{8} + N(\sigma_f^2)\frac{\Delta t^2}{2} & 0 & -N(\sigma_{Bf}^2)\frac{\Delta t^2}{2} & 0 \\ 0 & A(\sigma_f^2)\frac{\Delta t^2}{2} + N(\sigma_{Bf}^2)\frac{\Delta t^4}{8} & N(\sigma_f^2)\frac{\Delta t^3}{3} + N(\sigma_{Bf}^2)\frac{\Delta t^5}{20} & 0 & -N(\sigma_{Bf}^2)\frac{\Delta t^3}{6} & 0 \\ -N(\sigma_{Bw}^2)\frac{\Delta t^2}{2} & 0 & 0 & N(\sigma_{Bw}^2)\frac{\Delta t^2}{2} & 0 & 0 \\ 0 & -N(\sigma_{Bf}^2)\frac{\Delta t^2}{2} & -N(\sigma_{Bf}^2)\frac{\Delta t^3}{6} & 0 & N(\sigma_{Bf}^2)\Delta t & 0 \\ 0 & 0 & 0 & 0 & 0 & N(\sigma_{Bm}^2)\Delta t \end{bmatrix}$$

with tuning params $\sigma_w^2, \sigma_f^2, \sigma_{Bw}^2, \sigma_{Bf}^2, \sigma_{Bm}^2$

4) $\Phi = I + F\Delta t + \frac{1}{2}F^2\Delta t^2$, state transition function to calculate $P_{n+1|n} = \Phi P_{n|n}\Phi^T + Q_d$

Measurement/Correction Step

1) Magnetometer: Measures $\tilde{m}^b = C_i^b(q)\, m^i + B_m + n_m$ (noise), m^i pulled from WMM. Can calculate this with magnetic dip θ_{dip} so $m^i = \begin{bmatrix} \cos\theta_{dip} \\ 0 \\ \sin\theta_{dip} \end{bmatrix}$.

Our prediction for measurement is $\hat{\tilde{m}}^b = C_i^b(\hat{q})\, m^i$. Hence

$$\tilde{m}^b - \hat{\tilde{m}}^b = C_i^b(q)\, m^i + B_m + n_m - C_i^b(\hat{q})\, m^i$$

$$\Rightarrow \delta\tilde{m}^b = \left[I - [\alpha\times]\right] C_i^b(\hat{q})\, m^i + B_m + n_m - C_i^b(\hat{q})\, m^i$$

$$\Rightarrow \delta\tilde{m}^b = \left[\left[\left(C_i^b(\hat{q})\, m^i\right)\times\right] \; I_3\right] \begin{Bmatrix} \alpha \\ B_m \end{Bmatrix} + n_m$$

Rewrite $\delta\tilde{m}^b$ as innovation, $\begin{Bmatrix} \alpha \\ B_m \end{Bmatrix}$ as state and as h, we get

$$\text{innovation} = \delta\tilde{m}^b = \underbrace{\left[\left[\left(C_i^b(\hat{q})\, m^i\right)\times\right] \; 0_3 \; 0_3 \; 0_3 \; 0_3 \; I_3\right]}_{h} \delta x + \underbrace{\text{diag}\left(\sigma_m^2\right)}_{R}$$

Note that ES MEKF uses measurements to innovation of nominal state, cause can't measure error.

2) GPS: measure $\tilde{r}^i$, with $\hat{r} - \hat{\tilde{r}} = I\,\delta r + n_r$. Hence

$$\delta\tilde{r}^i = \underbrace{\left[0_3 \; 0_3 \; I_3 \; 0_3 \; 0_3 \; 0_3\right]}_{h} \delta x + \underbrace{\text{diag}\left(\sigma_{GPS}^2\right)}_{R} \quad \text{(assuming diag)}$$

Note that we can do the same with $\hat{v} - \hat{\tilde{v}} = I\,\delta v + n_v$ if GPS gives velocity with measurement model better than just differentiating position measurements (i.e. it adds new data).

Can add wind as param in measurement noise (not discussed here).

Now we get $K_n = P_{n|n} H^T (H P_{n|n} H^T + R)^{-1}$, $\delta x_{n|n} = \delta x_{n|n-1} + K_n(\text{innovation})$.
Only add to error correction if doing multiple measurements at once.
Finally, $P_{n|n} = (I - K_n H) P_{n,n-1}$

Q give expanded/non-simplified equation here?

Now update nominal state

$$\hat{q}_{n|n} = q_{n|n-1} \otimes \begin{bmatrix} 1 \\ \delta\alpha/2 \end{bmatrix}$$
$$\hat{r}_{n|n} = \hat{r}_{n,n-1} + \delta r$$
$$\hat{v}_{n|n} = \hat{v}_{n|n-1} + \delta v$$

Note that after each measurement we should set $\delta x = 0$. But if not directly updating/calibrating sensors with biases, then we should NOT reset bias terms in δx to 0, but save them and subtract biases from any measurement (gyro, acc, mag) in EKF code.

Next Steps

1) We assume Earth is inertial frame but should account for Coriolis acceleration, Schuler tuning, and plumb-bob gravity.

2) Pull m_i from WMM for long distance flights with GPS.

3) Accounting for wind by updating GPS measurement cov matrix.

4) Look into UKF, IEKF maybe?

5) GPS velocity correction (combine with altimeter?)

6) Test with dynamics model with less assumptions/keeping 2nd degree terms.

Appendix

1) Prove $\exp\left(\Omega(\vec{\theta})\right) = \cos(\|\vec{\theta}\|) I_4 + \frac{\sin(\|\vec{\theta}\|)}{\|\vec{\theta}\|}\Omega(\vec{\theta})$ where $\vec{\theta} = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$, $\|\vec{\theta}\| = \sqrt{x^2+y^2+z^2}$, $\exp(A) = \sum_{k=0}^{\infty} \frac{A^k}{k!}$ $\forall$ matrices A, and

$$\Omega(\vec{\theta}) = \begin{bmatrix} 0 & -\vec{\theta}^T \\ \vec{\theta} & -[\vec{\theta}\times] \end{bmatrix} = \begin{bmatrix} 0 & -x & -y & -z \\ x & 0 & z & -y \\ y & -z & 0 & x \\ z & y & -x & 0 \end{bmatrix}$$

Now notice, $\left(\Omega(\vec{\theta})\right)^2 = \begin{bmatrix} 0 & -x & -y & -z \\ x & 0 & z & -y \\ y & -z & 0 & x \\ z & y & -x & 0 \end{bmatrix}\begin{bmatrix} 0 & -x & -y & -z \\ x & 0 & z & -y \\ y & -z & 0 & x \\ z & y & -x & 0 \end{bmatrix}$

$$= \left(-x^2 - y^2 - z^2\right) I_4$$

$$\left(\Omega(\vec{\theta})\right)^2 = -\|\vec{\theta}\|^2 I_4 \quad \ldots \text{(*)}$$

Now $\exp\left(\Omega(\vec{\theta})\right) = \sum_{k=0}^{\infty} \frac{\left(\Omega(\vec{\theta})\right)^k}{k!}$

$$= \sum_{k=0}^{\infty} \frac{\left(\Omega(\vec{\theta})\right)^{2k}}{(2k)!} + \sum_{k=0}^{\infty} \frac{\left(\Omega(\vec{\theta})\right)^{2k+1}}{(2k+1)!} \quad \because \text{splitting odd and even terms}$$

$$= \sum_{k=0}^{\infty} \frac{\left(\left(\Omega(\vec{\theta})\right)^2\right)^k}{(2k)!} + \sum_{k=0}^{\infty} \frac{\left(\left(\Omega(\vec{\theta})\right)^2\right)^k}{(2k+1)!}\Omega(\vec{\theta})$$

$$= \sum_{k=0}^{\infty} \frac{\left(-\|\vec{\theta}\|^2 I_4\right)^k}{(2k)!} + \sum_{k=0}^{\infty} \frac{\left(-\|\vec{\theta}\|^2 I_4\right)^k}{(2k+1)!} \quad \because \text{(*)}$$

$$= \sum_{k=0}^{\infty} \frac{(-1)^k \|\vec{\theta}\|^{2k}}{(2k)!} I_4 + \sum_{k=0}^{\infty} \frac{(-1)^k \|\vec{\theta}\|^{2k}}{(2k+1)!}\Omega(\vec{\theta})$$

$$= \cos(\|\vec{\theta}\|) I_4 + \sum_{k=0}^{\infty} \frac{(-1)^k \|\vec{\theta}\|^{2k}}{(2k+1)!}\Omega(\vec{\theta}) \quad \because \text{by taylor series } \cos(x) = \sum_{k=0}^{\infty} \frac{(-1)^k x^{2k}}{2k!}$$

$$= \cos(\|\vec{\theta}\|) I_4 + \sum_{k=0}^{\infty} \frac{(-1)^k \|\vec{\theta}\|^{2k+1}}{(2k+1)!}\left(\frac{1}{\|\vec{\theta}\|}\right)\Omega(\vec{\theta})$$

$$= \cos(\|\vec{\theta}\|) I_4 + \frac{\sin(\|\vec{\theta}\|)}{\|\vec{\theta}\|}\Omega(\vec{\theta}) \quad \because \text{taylor series for } \sin(x) = \sum_{k=0}^{\infty} \frac{(-1)^k x^{2k+1}}{(2k+1)!}$$